

# SISTEMAS DE GESTIÓN EMPRESARIAL



## Modelos, campos, relaciones y seguridad en Odoo

Prof. Rosa Mª Romero

1

## ARQUITECTURA INTERNA DE ODOO

Odoo tiene una arquitectura en tres capas:

1. **Base de datos:** Utiliza PostgreSQL.
2. **Servidor Odoo:** Procesa la lógica de negocio y actúa como servidor web.
3. **Cliente web (SPA, Single Page Application):** Interfaz basada en navegador, con tres modos principales:
  - **Backend:** Para empleados y administradores.
  - **Frontend (sitio web):** Para clientes o usuarios externos, incluyendo eCommerce y portal.
  - **TPV (Punto de venta):** Interfaz táctil para ventas físicas.

Además del cliente web, Odoo permite la comunicación externa a través de **XML-RPC** y se pueden construir APIs mediante *web controllers* personalizados.

2

## ARQUITECTURA INTERNA DE ODOO

- La **capa de base de datos** es el nivel más bajo. Odoo se basa en la base de datos PostgreSQL mediante la que gestiona el almacenamiento y la persistencia de los datos.
- Los archivos binarios (documentos, imágenes, etc.), normalmente los almacena en un directorio llamado *.filestore*



3

## ARQUITECTURA INTERNA DE ODOO

- La **capa lógica** que reside en el servidor de Odoo es la responsable de las interacciones con la capa de datos (peticiones y recepción de datos).
- En ella se encuentra el motor de asignación relacional de objetos (**ORM**).
- Realiza los cálculos y validaciones y devuelve los datos al cliente. Odoo utiliza Python.



4

## ARQUITECTURA INTERNA DE ODOO

- La **capa de presentación (cliente web)** es la que muestra los datos y permite la interacción del usuario mediante un cliente que interactúa con la API de ORM a través de llamadas a procedimientos remotos (RPC). Odoo utiliza XML, JavaScript y Qweb.



5

## SISTEMA MODULAR

- Odoo es completamente modular. Todo se basa en módulos: el núcleo está en el módulo base, sobre el que se construyen los demás.
- Tras instalar Odoo, inicialmente solo se puede acceder al backend con funciones básicas.
- Para comenzar a trabajar se suelen instalar módulos esenciales como **Ventas, Compras, Contabilidad** o **CRM**.

6

## SISTEMA MODULAR

- Para ampliar o adaptar Odoo **no se modifica el código base**: se desarrollan módulos personalizados que pueden:
  - Añadir o modificar modelos de datos
  - Cambiar la interfaz de usuario
  - Alterar el comportamiento de negocio
- Estos módulos son instalables y desinstalables, lo que facilita una personalización limpia y reversible.

7

## FRAMEWORK DE DESARROLLO

- Odoo es también un framework RAD (Rapid Application Development), que permite construir aplicaciones complejas con menos esfuerzo.
- **Tecnologías necesarias:**
  - Python, XML, HTML, JavaScript
  - QWeb, JQuery, XML-RPC
  - Conocimientos de ORM y arquitectura web

8

## PRINCIPIOS TÉCNICOS CLAVE

- **ORM (Object Relational Mapping):**
  - Las clases en Python definen los modelos de datos.
  - No se diseña directamente la base de datos ni se escriben consultas SQL manuales.
- **Arquitectura MVC (Modelo - Vista - Controlador):**
  - **Modelos:** Clases en Python que heredan de `models.Model`, mapeadas a tablas PostgreSQL.
  - **Vistas:** Definidas en XML, transformadas dinámicamente a HTML por el framework JS del cliente web.
  - **Controladores:** Métodos Python que implementan la lógica de negocio, y opcionalmente definen endpoints web.
- **Multi-Tenencia:** Un único servidor puede alojar múltiples bases de datos (clientes diferentes).
- **Herramientas integradas:**
  - Generador de informes
  - Sistema de traducción multilenguaje

9

## LA BASE DE DATOS DE ODOO

**Odoo no tiene un modelo de base de datos estático.** Esto se debe a que su arquitectura se basa en un **ORM (Object-Relational Mapping)** que genera la estructura de la base de datos en función de los **modelos Python activos** en la instancia concreta.

Cada instalación de Odoo puede tener una estructura de base de datos distinta, dependiendo de los módulos instalados. El ORM se encarga de traducir las clases Python (`models.Model`) en tablas, columnas y relaciones en PostgreSQL.

10

## LA BASE DE DATOS DE ODOO

Por esta razón:

- **No existe un diagrama ER global** aplicable a todas las instalaciones.
- La base de datos de cada empresa puede tener **tablas diferentes**, incluso si parten de la misma versión de Odoo.
- Columnas, constraints o relaciones pueden variar según personalizaciones y herencias de modelos.

11

## Modelos clave predefinidos

A pesar de su estructura dinámica, Odoo incluye modelos base ampliamente utilizados y bien documentados, por ejemplo:

- **res.partner:** modelo central que representa clientes, proveedores y contactos.
- **sale.order:** representa los pedidos de venta.

Estos modelos están presentes en la mayoría de las implementaciones, aunque sus campos pueden ampliarse o modificarse dependiendo de los módulos instalados o desarrollos personalizados.

12



## Herramientas para explorar la base de datos

### • Modo desarrollador (Developer Mode):

- Se activa desde el backend.
- Permite ver el nombre técnico del modelo (model) y del campo (field) pasando el cursor sobre un campo en cualquier formulario.
- También se puede acceder al árbol de vistas, acciones y menús desde el panel de desarrollador.

### • Cliente PostgreSQL (psql):

- Ideal para consultas SQL directas a la base de datos.
- Permite explorar tablas, columnas y relaciones reales.
- Ejemplo útil: consultar clientes que no han realizado pedidos.

```
SELECT name FROM res_partner
WHERE id NOT IN (
    SELECT DISTINCT partner_id FROM sale_order
);
```

13

## Convenciones en los nombres en modelos

| Elemento            | Convención de nombres                                    |
|---------------------|----------------------------------------------------------|
| Modelo (Python)     | En minúsculas y separado por punto: module.model         |
| Nombre compuesto    | Se usa guion bajo: account.move                          |
| Tabla en PostgreSQL | El punto se reemplaza por guion bajo: account_move       |
| Campos en la tabla  | Derivan directamente de los atributos en la clase Python |

Ejemplo:

```
# Modelo en Python class SaleOrder(models.Model):
    _name = 'sale.order'
```

Esta clase "SaleOrder" se convierte en la tabla sale\_order en PostgreSQL.

14

## COMPOSICIÓN DE UN MÓDULO

- Odoo está construido bajo un diseño modular. Tanto el **servidor** como el **cliente web** están formados por módulos que amplían y heredan del módulo principal llamado base.
- Cada funcionalidad, vista, modelo o comportamiento adicional se implementa mediante un módulo, ya sea oficial, de terceros o personalizado.
- **No se recomienda modificar directamente el código de los módulos oficiales.**
- La forma recomendada de personalizar Odoo es mediante módulos propios

15

## Ejemplo práctico

Supongamos que una empresa considera innecesarios ciertos campos del formulario de productos (por ejemplo, código de referencia o tipo de consumo).

Podemos desarrollar un módulo que **modifique la vista del producto** y oculte esos campos.

- Si luego se decide que sí eran necesarios, simplemente **se desinstala el módulo** y Odoo restaurará la vista original sin dejar rastros.

Este comportamiento es posible porque, al reiniciar el servidor o actualizar un módulo, Odoo:

- Interpreta los archivos Python (modelos y lógica).
- Aplica los cambios al ORM y a la base de datos.
- Carga y actualiza las definiciones XML (vistas, menús, acciones, datos).

16



## QUÉ ES UN MODELO EN ODOO

- En Odoo cualquier dato es accesible mediante **modelos**.
- Los modelos constituyen las tablas de la BBDD.
- Cada **modelo** es un objeto que contiene información descriptiva en campos, información acerca de relaciones con otros objetos, así como de lógica de negocio
- Estos objetos se declaran mediante modelos Python, que se pueden consultar en **Ajustes / Técnico / Estructura de la base de datos / Modelos**, habiendo habilitado el Modo desarrollador en Ajustes generales Es la lista de todos los modelos disponibles en la base de datos.

17

## QUÉ ES UN MODELO EN ODOO

**Un modelo en Odoo representa una tabla de la base de datos.** Al heredar de `models.Model`, la clase adquiere capacidades automáticas como:

- Gestión del acceso a los datos mediante el ORM.
- Creación, lectura, actualización y eliminación (CRUD).
- Validación de integridad.
- Soporte para múltiples registros (recordsets) y operaciones vectorizadas.

18

## QUÉ ES UN MODELO EN ODOO

Cuando un módulo es instalado o actualizado, Odoo:

1. Interpreta las clases Python que heredan de `models.Model`.
2. Detecta los campos definidos (`fields.*`).
3. Genera o actualiza automáticamente las **tablas, columnas y constraints** en PostgreSQL.
4. Establece integridad referencial y otras reglas automáticamente (campos obligatorios, únicos, etc.).

19

## Componentes básicos de un modelo

```
from odoo import models, fields

class AModel(models.Model):
    _name = 'a.model'
    _description = 'Descripción opcional del modelo'

    name = fields.Char(
        string="Nombre",
        required=True,
        default='Nombre por defecto',
        readonly=True,
        compute="_compute_name_custom",
        store=True,
        inverse="_write_name",
        search="_search_function",
        help="Texto de ayuda para el usuario",
        translate=True,
        company_dependent=True,
        copy=True
    )
```

### Explicación del ejemplo

#### 1. Clase Python:

- AModel hereda de `models.Model`, lo que convierte la clase en un modelo de Odoo.

#### 2. Atributos del modelo:

- **\_name:** Nombre técnico del modelo (obligatorio). Este será el identificador usado por el ORM y la interfaz.
- **\_description:** Texto descriptivo del modelo (opcional).

#### 3. Campos (fields):

- **name = fields.Char(...):** Crea una columna en la base de datos del tipo cadena de texto.
- Cada tipo de campo (`Char`, `Integer`, `Boolean`, `Many2one`, etc.) hereda de `fields.Field` y tiene múltiples atributos configurables.

20

# Modelos

The screenshot shows the Odoo 'Ajustes' (Settings) interface. The top navigation bar includes 'Ajustes', 'Opciones generales', 'Usuarios y compañías', 'Traducciones', and 'Técnico'. The 'Técnico' tab is active. A dropdown menu is open, showing various technical settings. The 'Modelos' (Models) option is highlighted in yellow. Other options in the menu include 'Interfaz de usuario', 'Elementos del menú', 'Vistas', 'Vistas personalizadas', 'Filtros de usuario', 'Recorridos', 'Estructura de la base de datos', 'Precisión decimal', 'Activos', 'Campos', 'Selección de campos', 'Restricciones del modelo', 'Relaciones ManyToMany', 'Archivos adjuntos', 'Registro', 'Análisis de rendimiento de software', 'Automatización', 'Acciones planificadas', and 'Activadores de tareas programadas'.

21

# Modelos

The screenshot shows the 'Nuevo Modelo' (New Model) screen in Odoo. The top navigation bar includes 'Ajustes', 'Opciones generales', 'Usuarios y compañías', 'Traducciones', and 'Técnico'. The 'Técnico' tab is active. The 'Nuevo Modelo' button is highlighted. A search bar is present. Below the search bar is a table listing existing models.

| Modelo                                            | Descripción del modelo                                            | Tipo        | Modelo transitorio                  |
|---------------------------------------------------|-------------------------------------------------------------------|-------------|-------------------------------------|
| <input type="checkbox"/> _unknown                 | Desconocido                                                       | Objeto base | <input type="checkbox"/>            |
| <input type="checkbox"/> auth.passkey.key         | Clave de acceso                                                   | Objeto base | <input type="checkbox"/>            |
| <input type="checkbox"/> auth.passkey.key.create  | Crear una clave de acceso                                         | Objeto base | <input checked="" type="checkbox"/> |
| <input type="checkbox"/> auth.totp.rate.limit.log | Cantidad límite de registros de TOTP                              | Objeto base | <input checked="" type="checkbox"/> |
| <input type="checkbox"/> auth_totp.device         | Dispositivo de autenticación                                      | Objeto base | <input type="checkbox"/>            |
| <input type="checkbox"/> auth_totp.wizard         | Asistente de la configuración de la autenticación de dos factores | Objeto base | <input checked="" type="checkbox"/> |
| <input type="checkbox"/> avatar.mixin             | Avatar Mixin                                                      | Objeto base | <input type="checkbox"/>            |

Listado de modelos de Odoo

22

## Tipos de modelos

- Existen 3 tipos de modelos.
  - **Modelo (Model)**. Modelo estándar más común, almacenado en la BBDD
  - **Transitorios (TransientModel)**. Son utilizados para almacenar datos de forma temporal que se eliminarán periódicamente sin intervención del usuario
  - **Abstractos (AbstractModel)**. Utilizados para definir clases abstractas que pueden ser heredadas por múltiples modelos.

23

## Campos de los modelos

Los modelos contienen campos (principales y relacionales) con datos y atributos

### **Campos principales:**

- Campos **básicos**: contienen datos numéricos o de tipo texto, como *Integer*, *Boolean*, *Char*, *Float*
- Campos **reservados**. Creados por Odoo para su uso por el sistema, como *id*, *create\_date*, *create\_uid*, *write\_date*
- Campos **avanzados**: Odoo requiere un campo *name* en todos los modelos. Campos como *Binary*, *Html*, *Image*, *Monetary*, *Selection*, *Date*
- Campos **calculados**: toman su valor mediante una función, no de la BBDD. Se trata de un campo declarado de forma estándar pero que tiene entre sus argumentos uno llamado *compute*, que referencia a la función de cálculo definida en otra parte del código.

24

## ATRIBUTOS TIPO FIELDS EN MODELOS DE ODOO

- Los **campos (fields)** en Odoo son atributos especiales dentro de las clases modelo, y son los encargados de definir tanto la estructura de la base de datos como la forma en que los usuarios interactúan con los datos.
- Todos los campos se definen como instancias de las clases del módulo fields, y son **mapeados automáticamente por el ORM** (Object-Relational Mapping) a columnas, claves foráneas o relaciones en la base de datos PostgreSQL.

25

## Campos de datos simples

| TIPO DE CAMPO           | DESCRIPCIÓN                                                                                                   |
|-------------------------|---------------------------------------------------------------------------------------------------------------|
| <b>fields.Char</b>      | Texto corto                                                                                                   |
| <b>fields.Text</b>      | Texto largo (sin límite de caracteres)                                                                        |
| <b>fields.Integer</b>   | Número entero                                                                                                 |
| <b>fields.Float</b>     | Número decimal                                                                                                |
| <b>fields.Boolean</b>   | Campo de tipo verdadero/falso (checkbox)                                                                      |
| <b>fields.Date</b>      | Fecha sin hora                                                                                                |
| <b>fields.Datetime</b>  | Fecha con hora                                                                                                |
| <b>fields.Html</b>      | Campo de texto enriquecido (HTML)                                                                             |
| <b>fields.Binary</b>    | Archivos binarios (en base64). Usado para adjuntar documentos o imágenes (hasta Odoo 12)                      |
| <b>fields.Image</b>     | Campo especializado para imágenes (desde Odoo 13). Permite redimensionado automático (max_width, max_height). |
| <b>fields.Selection</b> | Campo que permite seleccionar entre varias opciones definidas como tuplas ((valor, etiqueta))                 |

26

## Ejemplo: Selección con lista y función

```
type = fields.Selection([
    ('1', 'Básico'),
    ('2', 'Intermedio'),
    ('3', 'Completado')
])
dynamic_selection = fields.Selection(selection='get_options')

def get_options(self):
    return [('x', 'Opción X'), ('y', 'Opción Y')]
```

27

## Atributos de los campos

Los campos admiten atributos:

- *string*. Etiqueta visible.
- *default*. Valor por defecto, estático o calculado.
- *required* (True). Obligatorio
- *help*. Tooltip de ayuda
- *readonly*. De "sólo lectura"
- *index*. Si el campo se indexa en la BBDD
- *compute*. Transforma en campo calculado
- *store* (True). Almacena el resultado de campos calculados

28



## ATRIBUTOS RELACIONALES EN MODELOS DE ODOO

Estos campos definen relaciones entre modelos. El ORM se encarga de gestionar automáticamente claves foráneas y tablas intermedias.

- **Many2one:** Define una relación *muchos a uno*. Se representa como una clave foránea en la base de datos. Un registro de este modelo apunta a un único registro del modelo relacionado.
- Ejemplo: varios empleados pertenecen al mismo departamento

29

## ATRIBUTOS RELACIONALES EN MODELOS DE ODOO

- **One2many:** Define una relación *uno a muchos*. Se representa como una clave foránea en la base de datos. Requiere que exista previamente el Many2one correspondiente en el otro modelo.
- Ejemplo: un cliente ha efectuado varios pedidos.

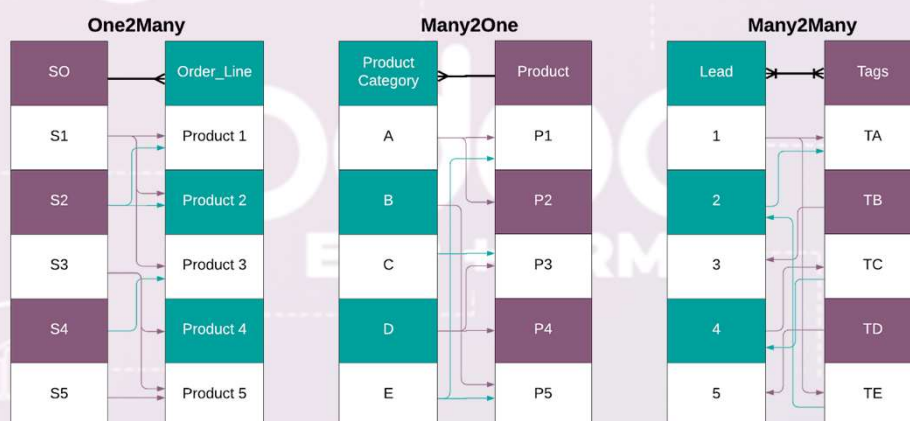
30

## ATRIBUTOS RELACIONALES EN MODELOS DE ODOO

- **Many2many:** Se utiliza cuando varios registros de un modelo pueden estar relacionados con varios registros de otro.
- Ejemplo, un estudiante puede estar matriculado en varios módulos, y cada módulo puede tener muchos estudiantes.
- Ejemplo: los clientes pueden comprar distintos productos y cada producto puede ser adquirido por varios clientes
- Odoo crea automáticamente una tabla intermedia donde guarda los pares de ids, que no aparece como modelo visible. A veces conviene definir manualmente esa tabla, especialmente cuando necesitamos varias relaciones entre los mismos modelos

31

## Campos relacionales de los modelos



32

## Campos especiales: related

El campo `related` permite mostrar en el modelo actual un campo de otro modelo con el que se tenga una relación, sin duplicar la información.

### Ejemplo:

```
pais_id = fields.Many2one('res.country', string="País")
bandera = fields.Image(related='pais_id.bandera', store=True)
```

Si se añade `store=True`, el valor se guarda físicamente en la base de datos, lo que permite búsquedas más rápidas, pero rompe la tercera forma normal. Odoo se encarga de mantener la coherencia.

33

## Campos especiales: related

Usos comunes:

- Mostrar campos de modelos relacionados en formularios.
- Tener versiones duplicadas del mismo campo (por ejemplo, imágenes en varias resoluciones).
- Usar el mismo campo con diferentes widgets.

34

## Otros tipos menos comunes

| Tipo de campo                   | Descripción                                                                             |
|---------------------------------|-----------------------------------------------------------------------------------------|
| <b>fields.Reference</b>         | Referencia genérica a <i>cualquier</i> modelo. Guarda el nombre del modelo y el ID.     |
| <b>fields.Many2oneReference</b> | Igual que Reference, pero solo permite selección desde una lista específica de modelos. |

### ERP + CRM

Estos campos son útiles para estructuras más genéricas o extensibles, aunque menos utilizados en la mayoría de los módulos estándar.

35

## Ejemplo

En este ejemplo práctico se definen dos modelos: Network (red) y PC (ordenador), que se relacionan de diferentes maneras.

```
# -*- coding: utf-8 -*-
from odoo import models, fields

class Network(models.Model):
    _name = 'networks.net'
    _description = 'Modelo de Redes'

    name = fields.Char()
    net_ip = fields.Char()
    mask = fields.Integer()
    net_map = fields.Image()
    net_class = fields.Selection([('a', 'Clase A'), ('b', 'Clase B'), ('c', 'Clase C')])
    pcs = fields.One2many('networks.pc', 'net_id', string="PCs")
    servers = fields.Many2many('networks.pc', relation='net_servers_rel', string="Servidores")

class PC(models.Model):
    _name = 'networks.pc'
    _description = 'Modelo de PCs'

    name = fields.Char(default="PC")
    number = fields.Integer()
    ip = fields.Char()
    ping = fields.Float()
    registered = fields.Date()
    uptime = fields.Datetime()
    net_id = fields.Many2one('networks.net', string="Red")
    user_id = fields.Many2one('res.partner', string="Usuario")
    servers = fields.Many2many('networks.net', relation='net_servers_rel', string="Servidores de red")
```

36

## Ejemplo

Por un lado, existe una relación One2many / Many2one entre ellos: cada red puede tener varios ordenadores conectados (pcs), pero cada PC pertenece a una única red (net\_id). Esto se consigue porque el modelo PC tiene un campo net\_id = fields.Many2one('networks.net'), y el modelo Network define el campo inverso pcs = fields.One2many('networks.pc', 'net\_id').

Gracias a esta relación, Odoo puede mostrar todos los PCs vinculados a una red dentro de su vista, y también permite navegar desde un PC hasta su red asociada.

```
# -*- coding: utf-8 -*-
from odoo import models, fields

class Network(models.Model):
    _name = 'networks.net'
    _description = 'Modelo de Redes'

    name = fields.Char()
    net_ip = fields.Char()
    mask = fields.Integer()
    net_map = fields.Image()
    net_class = fields.Selection([('a', 'Clase A'), ('b', 'Clase B'), ('c', 'Clase C')])
    pcs = fields.One2many('networks.pc', 'net_id', string="PCs")
    servers = fields.Many2many('networks.pc', relation='net_servers_rel', string="Servidores")

class PC(models.Model):
    _name = 'networks.pc'
    _description = 'Modelo de PCs'

    name = fields.Char(default="PC")
    number = fields.Integer()
    ip = fields.Char()
    ping = fields.Float()
    registered = fields.Date()
    uptime = fields.Datetime()
    net_id = fields.Many2one('networks.net', string="Red")
    user_id = fields.Many2one('res.partner', string="Usuario")
    servers = fields.Many2many('networks.net', relation='net_servers_rel', string="Servidores de red")
```

37

## Ejemplo

Por otro lado, ambos modelos comparten una relación **Many2many** llamada servers, que representa los **servidores disponibles en una red y las redes a las que pertenece un servidor**.

Aquí se define manualmente el nombre de la tabla intermedia net\_servers\_rel, lo que asegura que ambos lados de la relación (Network.servers y PC.servers) apunten a la misma tabla y no se creen relaciones duplicadas. Esto es útil cuando se quiere mantener una correspondencia exacta entre los registros y un control total sobre el modelo relacional.

```
# -*- coding: utf-8 -*-
from odoo import models, fields

class Network(models.Model):
    _name = 'networks.net'
    _description = 'Modelo de Redes'

    name = fields.Char()
    net_ip = fields.Char()
    mask = fields.Integer()
    net_map = fields.Image()
    net_class = fields.Selection([('a', 'Clase A'), ('b', 'Clase B'), ('c', 'Clase C')])
    pcs = fields.One2many('networks.pc', 'net_id', string="PCs")
    servers = fields.Many2many('networks.pc', relation='net_servers_rel', string="Servidores")

class PC(models.Model):
    _name = 'networks.pc'
    _description = 'Modelo de PCs'

    name = fields.Char(default="PC")
    number = fields.Integer()
    ip = fields.Char()
    ping = fields.Float()
    registered = fields.Date()
    uptime = fields.Datetime()
    net_id = fields.Many2one('networks.net', string="Red")
    user_id = fields.Many2one('res.partner', string="Usuario")
    servers = fields.Many2many('networks.net', relation='net_servers_rel', string="Servidores de red")
```

38

## CAMPOS COMPUTADOS

Los modelos en Odoo permiten definir campos (fields) con distintos comportamientos, además de almacenar datos simples o relacionales. Estos son los **campos computados o calculados (compute)**

Estos campos no almacenan un valor fijo en la base de datos, sino que calculan su valor dinámicamente en función de otros campos.

39

## CAMPOS COMPUTADOS

```

taken_seats = fields.Float(string="Plazas ocupadas", compute="_compute_taken_seats")

@api.depends('seats', 'attendee_ids')
def _compute_taken_seats(self):
    for record in self:
        if not record.seats:
            record.taken_seats = 0.0
        else:
            record.taken_seats = 100.0 * len(record.attendee_ids) / record.seats

```

- `compute="_compute_taken_seats"`: indica la función que calcula el valor.
- `@api.depends(...)`: Odoo vuelve a calcular el campo **cuando cambien** los campos indicados.
- Se accede a `self` como recordset, por eso se debe iterar (`for record in self:`).
- Este campo no se guarda en la base de datos a menos que se indique lo contrario.

40



## CAMPOS COMPUTADOS

```

taken_seats = fields.Float(string="Plazas ocupadas", compute="_compute_taken_seats")

@api.depends('seats', 'attendee_ids')
def _compute_taken_seats(self):
    for record in self:
        if not record.seats:
            record.taken_seats = 0.0
        else:
            record.taken_seats = 100.0 * len(record.attendee_ids) / record.seats

```

### Guardar campos computados en la base de datos

```

taken_seats = fields.Float(compute="_compute_taken_seats", store=True)

```

- Si se usa store=True, el valor se **almacena en la base de datos**.
- sto permite hacer búsquedas y ordenaciones, pero **solo se actualizará** si los campos en @api.depends() cambian.
- Si no se depende de otros campos, usar store=True **puede generar inconsistencias**.

41

## CAMPOS COMPUTADOS

### Búsqueda sobre campos compute (sin store=True)

Para permitir búsquedas en campos no almacenados, se puede definir una función de búsqueda:

```

taken_seats = fields.Float(compute="_compute_taken_seats",
                           search="_search_taken_seats")

def _search_taken_seats(self, operator, value):
    return [('attendee_ids', '!=', False)] # Devuelve un dominio

```

Las búsquedas sobre campos computados sin store=True son limitadas y pueden afectar el rendimiento.

42

## CAMPOS COMPUTADOS

En caso de necesitar mostrar un valor calculado, pero también almacenarlo y buscarlo, se pueden usar dos campos. En la vista XML el campo guardado se puede hacer invisible.

```
# Campo visible (no guardado)
visible_field = fields.Float(compute="_compute_value", store=False)

# Campo invisible (guardado)
stored_field = fields.Float(default=0.0)

def _compute_value(self):
    for rec in self:
        computed = some_calculation()
        rec.visible_field = computed
        rec.stored_field = computed # Se guarda el valor real
```

```
<field name="stored_field" invisible="1"/>
```

43

## CAMPOS COMPUTADOS

En un campo computado no se puede escribir un valor, dado que se genera automáticamente.

En caso de que queramos permitir la escritura manual, se puede utilizar el parámetro `inverse`:

```
price = fields.Float(compute='_compute_price', inverse='_set_price')

def _compute_price(self):
    for record in self:
        record.price = record.cost + record.margin

def _set_price(self):
    for record in self:
        record.cost = record.price - record.margin
```

44

## VALORES POR DEFECTO

- Se pueden asignar valores por defecto a los campos de un modelo usando el parámetro `default`.
- Si el valor es **estático**, se asigna directamente.
- Si el valor es **dinámico**, se usa:
  - Una **función lambda** (función anónima en una línea).
  - Una **función normal** (debe estar definida antes de ser usada).
- Odoo **interpreta todos los archivos Python al arrancar el servidor**. Por eso:
  - Si definimos `default=fields.Date.today()`, se ejecutará solo una vez (al arrancar).
  - Si definimos `default=lambda self: fields.Date.today()`, se ejecuta cada vez que se crea un registro nuevo.
- **Recomendación:** usar siempre lambda o funciones para valores dinámicos.

45

## RESTRICCIONES (CONSTRAINTS)

Las restricciones permiten **validar los datos antes de guardar** un registro. Se pueden definir de dos formas:

- **Restricciones en Python: `@api.constraints`:**
  - Se ejecuta al guardar el registro.
  - Puede validar varios campos a la vez
  - Impide el guardado si se lanza una excepción `ValidationError`
- **Restricciones SQL: `_sql_constraints`**
  - Más eficientes para restricciones simples, como valores únicos o no nulos
  - Se definen como una lista de tuplas con:
    - Nombre de la restricción
    - Código SQL
    - Mensaje de error

46

# RESTRICCIONES (CONSTRAINTS)

```
from odoo.exceptions import ValidationError

@api.constrains('age')
def _check_age(self):
    for record in self:
        if record.age > 20:
            raise ValidationError(f"Demasiado viejo: {record.age}")
```

```
_sql_constraints = [
    ('unique_name', 'unique(name)', 'El nombre debe ser único.'),
    ('positive_price', 'CHECK(price >= 0)', 'El precio debe ser positivo.')
]
```

47

Ejemplo  
completo:  
computed,  
default y  
constraint

```
from odoo import models, fields, api
from odoo.exceptions import ValidationError

class Event(models.Model):
    _name = 'event.management'

    name = fields.Char(default="Nuevo evento")
    start_date = fields.Date(default=lambda self: fields.Date.today())
    attendees = fields.Many2many('res.partner', string="Asistentes")
    seats = fields.Integer(default=50)
    taken_seats = fields.Float(compute="_compute_taken", store=True)

    @api.depends('seats', 'attendees')
    def _compute_taken(self):
        for r in self:
            if not r.seats:
                r.taken_seats = 0.0
            else:
                r.taken_seats = 100.0 * len(r.attendees) / r.seats

    @api.constrains('seats')
    def _check_seats(self):
        for r in self:
            if r.seats < 0:
                raise ValidationError("El número de asientos no puede ser negativo.")
```

48

## SEGURIDAD EN MODELOS ODOO

Odoo gestiona la seguridad a nivel de modelo utilizando **Listas de Control de Acceso (ACL)**, que permiten definir qué operaciones puede realizar cada grupo de usuarios sobre cada modelo. Estas operaciones son:

- perm\_read: Lectura
- perm\_write: Escritura
- perm\_create: Creación
- perm\_unlink: Eliminación
- Estas ACL se definen en el archivo `security/ir.model.access.csv`, que debe estar incluido dentro del fichero `__manifest__.py` del módulo

```
'data': [
    'security/ir.model.access.csv',
],
```

49

## Estructura del archivo ir.model.access.csv

- Este archivo es un CSV que debe comenzar con una **cabecera obligatoria**, seguida de líneas que definen los permisos para cada modelo y grupo.

- Formato: `id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink`

- Ejemplo:

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
acl_mi_modulo,Permisos modelo mi módulo, mdeolo_mi_modulo,base.group_user,1,1,1,1
```

50

## Estructura del archivo ir.model.access.csv

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
acl_mi_modulo,Permisos modelo mi módulo, mdeolo_mi_modulo,base.group_user,1,1,1,1
```

### Explicación de los campos:

- **id** → Identificador interno de la regla.
- **name** → Nombre descriptivo (opcional para uso interno).
- **model\_id:id** → Referencia externa al modelo, por convención: `model_<nombre_modelo>`.
- **group\_id:id** → Grupo al que se aplica la ACL. `base.group_user` aplica a todos los usuarios normales.
- **perm\_read, perm\_write, perm\_create, perm\_unlink** → Valores 1 (permitido) o 0 (denegado).

Si queremos que el modelo sea accesible para el administrador sin restricciones, también debes definir una ACL específica para el grupo `base.group_system` o no limitar el acceso.

51

## Definir grupos personalizados

Si necesitamos que ciertos permisos estén restringidos a un grupo específico (por ejemplo, "Bibliotecario"), primero debemos crear ese grupo. Esto se hace con un archivo XML que también debe incluirse en el `__manifest__.py`.

```
'data': [
    'security/groups.xml',
    'security/ir.model.access.csv',
],
```

Ejemplo de grupo (security/groups.xml):

```
<?xml version="1.0" encoding="UTF-8"?>
<odoo>
  <record id="grupo_bibliotecario" model="res.groups">
    <field name="name">Bibliotecario</field>
    <field name="users" eval="[(4, ref('base.user_admin'))]" />
  </record>
</odoo>
```

Este ejemplo crea un grupo llamado **"Bibliotecario"** y añade al usuario administrador (`base.user_admin`) como miembro por defecto.

52



## Definir grupos personalizados

Una vez definido el grupo, puedes usar su id para asignarle permisos en el archivo de ACL:

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
acl_biblio_lectura,Permisos lectura para
bibliotecarios,model_biblio_libro,modulo_grupo_bibliotecario,1,0,0,0
acl_biblio_total,Permisos completos,model_biblio_libro,base.group_system,1,1,1,1
```

Reemplazaremos `modulo_grupo_bibliotecario` con el nombre técnico de nuestro módulo.

53

## Buenas prácticas

- Crear siempre **grupos separados** para roles funcionales personalizados.
- Usar el grupo `base.group_system` para permitir acceso total a administradores.
- Asegurarse de tener al menos una ACL para cada modelo, o **no será accesible** desde el backend.
- **Evita dar permisos amplios a `base.group_public`** (usuarios no autenticados).
- Mantener separado el archivo de grupos (`groups.xml`) y el de permisos (`ir.model.access.csv`) para mayor claridad.

54

## ENLACES DE INTERÉS

- [https://www.odoo.com/documentation/19.0/es\\_419/contributing/development/coding\\_guidelines.html](https://www.odoo.com/documentation/19.0/es_419/contributing/development/coding_guidelines.html)
- Canal de Youtube de Josuhe

